

Onboarding Guide

Set up your machine, load the plugins, and ship work through the agent pipeline — from your desk or from anywhere.

Audience	Scope	Read time
Senior engineers	Setup · Plugins · Permissions · Daily workflow	~15 minutes

Internal engineering document. Quetrex is a framework layered on Claude Code — compute is bring-your-own.

CONTENTS

What's in this guide

Read §1 for the mental model, then work top-to-bottom. §7 is a one-page reference to keep open.

- 1 What Quetrex is — the mental model
- 2 One-time machine setup — Tailscale, SSH, Claude Code auth
- 3 Load & use the plugins — marketplace, pinning, cloud parity
- 4 Permissions — run AUTO mode, not skip-permissions
- 5 The daily workflow — the lean command set
- 6 Work from anywhere — SSH or `claude.ai/code`
- 7 Quick reference — commands, plugins, gotchas

THE ONE-SENTENCE VERSION

Quetrex is a private plugin family that turns Claude Code into a pipeline of specialist agents whose guarantees are enforced by on-disk hooks — and you run it under *your own* Claude subscription.

SECTION 1

What Quetrex is

The mental model. Get this and everything else is mechanics.

Quetrex is a **private plugin family for Claude Code**. It doesn't replace Claude and it doesn't host any AI — it's the framework that makes a fleet of agents produce reviewable, mergeable work instead of a wall of confident prose.

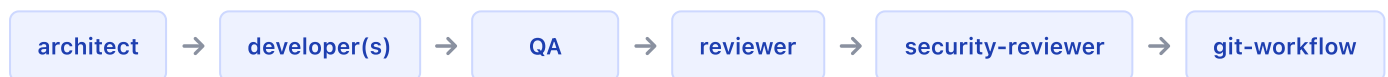
The engine + per-stack packs

There is one stack-agnostic engine and a set of thin, stack-specific packs on top of it:

- **quetrex-factory** — the ENGINE. Stack-agnostic. Defines the agents, the pipeline, and the hooks. Goes on *every* repo.
- **Per-stack packs** — `quetrex-nextjs`, `quetrex-python`, `quetrex-rust`, `quetrex-swift`. These add the build/lint/test/verify specifics for one stack. You enable *only* the pack that matches the repo.

The pipeline it enforces

Every task runs through a fixed sequence of specialist agents. Each stage has one job and hands off to the next:



The architect plans and assigns strict file ownership; developers work in parallel worktrees on separate sub-branches; QA proves green with real exit codes; the reviewer reads the full diff for logic and architecture; the security-reviewer scans for vulnerabilities; git-workflow opens the squash PR.

The load-bearing idea: guarantees are hooks, not prose

This is what makes Quetrex trustworthy. **Every guarantee is a HOOK that reads an on-disk artifact — never an agent's word for it.** An agent claiming "tests pass" means nothing; a hook reading the actual exit code and the actual test output on disk is the gate. You cannot talk your way past a hook.

WHY THIS MATTERS

The pipeline's promises (green build, no secrets committed, branch discipline, no force-push) survive even when an agent is wrong or over-eager, because they're mechanically checked against files — not asserted in chat.

The kanban

Coordination — tasks, status columns, escalations — lives on a shared **kanban**. It's your window into every task across every machine. You watch the board; the pipeline does the work.

Compute is bring-your-own

Quetrex is the framework, *not* the AI. Each engineer runs Claude Code under **their own Claude subscription** (Max or Pro). There is no shared inference bill and no central runner burning tokens on your behalf — your machine (or your cloud session) runs the agents on your plan.

Quetrex provides Plugins, agents, hooks, the kanban, the command set.

You provide Your Claude Code subscription and the machine (or cloud session) it runs on.

SECTION 2

One-time machine setup

Do this once per machine. It makes your always-on box reachable from anywhere and gets Claude Code authenticated.

2.1 Tailscale — a stable name for every machine

Tailscale puts all your machines on one private mesh network (a *tailnet*). Every machine gets a stable name reachable from anywhere — no port-forwarding, no VPN concentrator, no firewall spelunking.

1. Install Tailscale from `tailscale.com/download`.
2. Sign in with the **company account** so you join the shared tailnet.
3. Bring the machine up:

```
# connect this machine to the tailnet
tailscale up

# find machine names / IPs on your tailnet
tailscale status
```

`tailscale status` lists each machine's tailnet name (e.g. `studio`) and its `100.x.y.z` address. That name is stable — note it for the next step.

2.2 SSH config — `ssh studio` from anywhere

Add a `Host` block to `~/.ssh/config` so a short alias connects to your always-on machine over the tailnet. This is how you drive terminal sessions from a laptop, a café, or a phone.

```
# ~/.ssh/config
Host studio
  HostName studio          # tailscale name (or 100.x.y.z)
  User    you              # your username on that box
```

Now `ssh studio` lands you on your always-on machine. On flaky networks (mobile, hotel wifi) use `mosh` instead — it survives roaming and dropped links and keeps your session alive:

```
mosh studio
```

2.3 Claude Code auth

Log in once with your Max (or Pro) subscription. This binds Claude Code on this machine to *your* plan — the compute you'll run the pipeline on.

```
claude          # launch Claude Code
/login          # authenticate with your Claude subscription
```

2.4 AUTO permission mode

The last piece of machine setup is enabling AUTO permission mode so the pipeline runs without babysitting — and doing it correctly. It's important enough to have its own section: **see §4** before you flip it on.

SETUP CHECKLIST

Tailscale up & joined the company tailnet · `~/.ssh/config` Host block added · `claude /login` done
under your subscription · AUTO mode configured (§4).

SECTION 3

Load & use the plugins

Add the private marketplace once; each repo pins what it needs. Identical locally and in the cloud.

3.1 Add the private marketplace (once)

Register the Quetrex plugin marketplace on your machine a single time:

```
/plugin marketplace add Glori-Holdings/quetrex-plugins
```

The marketplace is **private**. It's reached through your own GitHub access combined with the cloud GitHub proxy — so if you can read the org on GitHub, the marketplace resolves. No separate credentials to manage.

3.2 A project pins what it needs

You don't manually enable plugins per repo from memory. Each project **pins** its plugins via a checked-in `.claude/settings.json`. Clone the repo, and the right plugins are already declared:

```
// .claude/settings.json (checked into the repo)
{
  "extraKnownMarketplaces": {
    "quetrex": {
      "source": {
        "source": "github",
        "repo": "Glori-Holdings/quetrex-plugins"
      }
    }
  },
  "enabledPlugins": {
    "quetrex-factory@quetrex": true,
    "quetrex-nextjs@quetrex": true
  }
}
```

THE RULE

The **engine** (`quetrex-factory`) goes on every repo. The **stack pack goes only on repos of that stack** — a Next.js repo enables `quetrex-nextjs`, a Rust repo enables `quetrex-rust`, and never both.

3.3 Local and cloud are identical

This whole mechanism works the same way whether you run Claude Code on your own machine or in **claude.ai/code** (the cloud sandbox). In the cloud, the session:

1. clones the repo,
2. installs the pinned plugins from the private marketplace, and
3. runs the hooks in the sandbox — exactly as they'd run locally.

So **"work from anywhere" = open claude.ai/code**. There is no self-hosted runner to stand up or maintain; the cloud session runs under your own subscription with the same guarantees.

3.4 Updating plugins

Keep plugins current with the native command — nothing Quetrex-specific:

```
/plugin update
```

DON'T

There is no `/quetrex-update`. Use the native `/plugin update` to pull new plugin versions.

Permissions — get this right

Run AUTO mode. Do not reach for `--dangerously-skip-permissions`. The difference is the whole safety story.

4.1 AUTO mode vs. skip-permissions

	AUTO MODE (USE THIS)	<code>--DANGEROUSLY-SKIP-PERMISSIONS</code>
Approval prompts	None — full autonomy, no constant clicking	None
Destructive-action guard	A documented classifier blocks destructive actions	Nothing blocks anything
Guard hooks	Still fire (deny-guard, secret-scan, etc.)	Bypassed intent — for throwaway envs only
When to use	Your daily driver	Throwaway VMs only

AUTO mode gives you the autonomy you want (no prompt-fatigue) *and* keeps the safety net: a documented classifier refuses destructive operations, and the guard hooks continue to run. `--dangerously-skip-permissions` is documented for disposable VMs only — do not use it for real work.

4.2 Enable AUTO mode

Set the default mode in **your own** `~/.claude/settings.json` :

```
// ~/.claude/settings.json (YOUR user settings)
{
  "permissions": {
    "defaultMode": "auto"
  }
}
```

WHY IT MUST BE USER SETTINGS

`auto` is honored **only in USER settings or managed settings** — a project cannot grant itself auto mode. This is deliberate: a repo you clone can't silently escalate its own autonomy.

With that set, plain `claude` starts in auto mode. To switch modes mid-session, **Shift+Tab cycles** through the permission modes — look for the `auto mode on` indicator to confirm you're in it.

4.3 Whitelisting trusted infra

The classifier is conservative and will sometimes block routine operations against infrastructure it doesn't recognize. Add trusted hosts/targets to `autoMode.environment` so routine ops flow without tripping the guard:


```
{
  "permissions": { "defaultMode": "auto" },
  "autoMode": {
    "environment": [ /* trusted infra the classifier should allow */ ]
  }
}
```

4.4 The guardrails you can't ship past

These are HOOKS. They fire in auto mode regardless of what any agent decides, because they read on-disk state — not agent claims:

HOOK	GUARANTEES
deny-guard	Blocks destructive / disallowed operations.
secret-scan	Stops secrets from being committed.
enforce-branch	Keeps work off <code>main</code> — feature branches only.
verify-gate	Requires a real green build/test result before a PR is considered.
merge-gate	Prevents force-pushing / merging past review.

BOTTOM LINE

In AUTO mode you get hands-off autonomy, but you **can't ship broken code or force-push past the gates** — the hooks won't let you.

SECTION 5

The daily workflow

A lean command set. Set up once, then it's new → refine → build, and you observe the board.

5.1 Setup commands

<code>/quetrex-setup</code>	Once per machine. GitHub auth, git identity, API keys.
<code>/quetrex-init</code>	Once per repo. Auto-detects the GitHub remote and links the kanban project.

5.2 The per-task loop

COMMAND	WHAT IT DOES
<code>/quetrex-task-new</code>	Create a task on the kanban.
<code>/quetrex-task-refine</code>	Sharpen the task into a buildable spec, grounded in the repo. The highest-leverage step — a sharp spec is the difference between a clean build and rework.
<code>/quetrex-task-build</code>	Run the full pipeline (architect → ... → git-workflow).
<code>/quetrex-task-rework</code>	For stuck or failed tasks: agree a fix plan and re-queue through the pipeline.

SPEND YOUR ATTENTION HERE

`/quetrex-task-refine` is where a human adds the most value. Under-specified tasks are the top cause of rework — invest in the spec, then let the pipeline run.

5.3 Merge is automatic

You do **not** merge routine tasks by hand. After the pipeline's review:

- **Clean review → auto-merges.** No action from you.
- **Issues found → rework.** The task loops back automatically.
- **Risky → escalates to a human.** You merge it on GitHub after a look.

DON'T

There is no `/quetrex-task-merge` for routine work — merge is automatic. You only touch a merge when review escalates it to you.

5.4 Deploy & complete

Only **production deploy is manual** — the one deliberate human gate at the end:

<code>/quetrex-deploy</code>	Deploy to production. Manual, on purpose.
<code>/quetrex-task-complete</code>	Mark a deployed task complete.

5.5 Observe, don't babysit

Watch the **kanban**, not your terminals. Your job is to act at gates — review escalations and production deploys — and to respond when a task raises a `needs_clarity` escalation asking a question only you can answer. Everything between those points runs itself.

SECTION 6

Work from anywhere

Two paths to the same pipeline. The board is your window from any device.

PATH A — SSH TO YOUR ALWAYS-ON MACHINE

Your desktop/studio box stays on and runs the agents under your subscription. Reach it over the tailnet from any device:

```
ssh studio          # or:
mosh studio         # flaky networks
```

Tailscale gives it a stable name; `~/ .ssh/config` gives you the short alias (§2). Your laptop is just a terminal — the compute lives on the always-on box.

PATH B — CLAUDE.AI/CODE (CLOUD)

No always-on machine needed. Open **claude.ai/code**; the cloud session clones the repo, installs the pinned plugin from the private marketplace, and runs the hooks in the sandbox — under *your* subscription.

Same plugins, same gates, same guarantees as local. Nothing to self-host.

YOUR WINDOW INTO EVERYTHING

Whichever path you take, the **kanban board** shows every task's status across every machine and cloud session, from your phone or a browser tab. You steer from the board and act only at the gates.

Because compute is bring-your-own and the plugin loads identically in both places, there's no difference in output between working at your desk, over SSH from a hotel, or from a cloud session on a borrowed laptop.

Quick reference

Keep this page open. The commands, the plugins, and the traps.

THE COMMAND SET

<code>/quetrex-setup</code>	Once / machine
<code>/quetrex-init</code>	Once / repo
<code>/quetrex-task-new</code>	Create task
<code>/quetrex-task-refine</code>	Sharpen spec ★
<code>/quetrex-task-build</code>	Run pipeline
<code>/quetrex-task-rework</code>	Fix + re-queue
<code>/quetrex-deploy</code>	Prod deploy (manual)
<code>/quetrex-task-complete</code>	Mark done

★ highest-leverage step. Native `/plugin update` updates plugins.

THE PLUGINS

<code>quetrex-factory</code>	Engine — every repo
<code>quetrex-nextjs</code>	Next.js repos
<code>quetrex-python</code>	Python repos
<code>quetrex-rust</code>	Rust repos
<code>quetrex-swift</code>	Swift repos

Engine everywhere; exactly one stack pack per repo, pinned in `.claude/settings.json`.

THE PIPELINE

architect → developer(s) → QA → reviewer → security-reviewer → git-workflow

Key gotchas

1 • RUN AUTO MODE, NOT SKIP-PERMISSIONS

`--dangerously-skip-permissions` is for throwaway VMs only. AUTO mode keeps the classifier and guard hooks live while removing prompts. Set `"defaultMode": "auto"` in **your own** `~/.claude/settings.json` — a project can't grant auto.

2 • MERGE IS AUTOMATIC

No `/quetrex-task-merge` for routine work. Clean reviews auto-merge; only escalated/risky PRs need your hand on GitHub. Only production deploy is manual.

3 • UPDATE WITH NATIVE /PLUGIN UPDATE

There is no `/quetrex-update`. Use `/plugin update`.

4 • ONE STACK PACK PER REPO

`quetrex-factory` on every repo; enable only the stack pack that matches. Never two stack packs on the same repo.

REMEMBER

Compute is bring-your-own — Quetrex is the framework, your Claude subscription is the engine. Guarantees are hooks reading files, not agents' promises. Observe the board; act only at the gates.